

ANSIBLE

Practice Course

7 Comprehensive Lab Workbook

Lab 1 — YAML Syntax
Lab 2 — Using Variables
Lab 3 — Ansible Playbooks
Lab 4 — Ansible Modules
Lab 5 — Conditionals
Lab 6 — Loops
Lab 7 — Ansible Roles

LAB 1

YAML Files

A comprehensive guide to YAML syntax as used in Ansible playbooks, inventories, and variable files

Learning Objectives

- Understand the structure and rules of YAML syntax
- Identify and use scalar, sequence, and mapping data types
- Write multi-line strings using literal and folded block scalars
- Avoid and fix common YAML syntax errors
- Create valid YAML inventory files and variable files for Ansible

1. What Is YAML?

YAML (YAML Ain't Markup Language) is a human-readable data serialization format used throughout Ansible. Every playbook, inventory, variable file, and role configuration is written in YAML. It is designed to be easy to read and write by humans while remaining unambiguous for machines to parse.

Three fundamental rules govern YAML syntax:

- Indentation uses spaces, never tabs — YAML is whitespace-sensitive
- It is case-sensitive: 'Name' and 'name' are different keys
- Structure is conveyed through indentation level, not brackets or braces

2. YAML Data Types

Scalars

Scalars are single values: strings, integers, floats, booleans, and null.

```
# Strings — quotes are optional unless the value is ambiguous
name: web-server
message: 'Hello, World!'
description: "This is a multi-word value"

# Integers and floats
port: 80
timeout: 30
ratio: 1.5

# Booleans — all forms below are valid in YAML 1.1 (used by Ansible)
enabled: true      # or: yes, on, True
disabled: false    # or: no, off, False
```

```
# Null
value: null
empty:
```

⚠ Boolean Gotcha

YAML 1.1 (used by Ansible) treats yes, no, on, off, true, false as booleans.
If you need the literal string 'yes', quote it: answer: 'yes'

Sequences (Lists)

Sequences represent ordered lists of items. Each item is prefixed with a dash and a space.

```
# Block sequence (preferred in Ansible)
packages:
  - nginx
  - git
  - curl

# Inline (flow) sequence - same data, compact form
packages: [nginx, git, curl]

# List of dictionaries - very common in task lists
users:
  - name: alice
    uid: 1001
  - name: bob
    uid: 1002
```

Mappings (Dictionaries)

Mappings are key-value pairs. They are the most common structure in Ansible playbooks.

```
# Block mapping
database:
  host: db.example.com
  port: 5432
  name: myapp_db

# Inline (flow) mapping
database: {host: db.example.com, port: 5432}

# Nested mappings
server:
  web:
    port: 80
    ssl: true
  db:
    port: 5432
```

3. Multi-Line Strings

YAML provides two block scalar styles for multi-line strings — essential for embedding scripts or file content in playbooks.

```
# Literal block scalar ( | ) - preserves newlines exactly
script_content: |
  #!/bin/bash
  echo 'Starting service'
  systemctl start nginx
  echo 'Done'

# Folded block scalar ( > ) - newlines become spaces (paragraph style)
description: >
  This is a long description that wraps
  across multiple lines in the file,
  but will be a single line when read.

# Strip trailing newlines with |- or >-
inline_script: |-
  echo 'no trailing newline'
```

4. Comments

```
# This is a full-line comment
host: web1 # This is an inline comment

# Comments can appear anywhere - they are ignored by the parser
# Use them liberally to document your intent
```

5. Quoting Rules

Most strings do not need quotes in YAML. However, quoting is required or strongly recommended in these cases:

Situation	Example	Solution
Value starts with special char	: value or - value	Quote: ': value'
Value is ambiguous boolean	value: yes	Quote: value: 'yes'
Value contains a colon+space	url: http://host: value	Quote: "http://host: value"
Value contains newlines/tabs	msg: line1\nline2	Double-quote: "line1\nline2"
Empty string	(nothing)	Quote: value: ""

6. A Complete Ansible YAML Example

The following example shows the full structure of an Ansible playbook with YAML features encountered most often:

```

---
# Mandatory document-start marker (optional but recommended)
- name: Configure web servers
  hosts: web_servers
  become: true

vars:
  http_port: 80
  server_name: web.example.com
  allowed_ips:
    - 10.0.0.1
    - 10.0.0.2
  db_config:
    host: db.example.com
    port: 5432
    name: myapp

tasks:
  - name: Install nginx
    apt:
      name: nginx
      state: present

  - name: Deploy nginx config
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
      notify: Restart nginx

handlers:
  - name: Restart nginx
    service:
      name: nginx
      state: restarted
...
# Optional document-end marker

```

7. Common YAML Errors

Error	Wrong	Correct
Tab character used for indentation	<TAB>key: value	key: value (spaces)
Inconsistent indentation	key:\n val: x	key:\n val: x
Missing space after colon	key:value	key: value
Missing space after dash	-item	- item
Unquoted colon in value	url: http://host:80	url: "http://host:80"
Duplicate key	host: web1\nhost: web2	hosts: [web1, web2]

Validation Tip

Use `yamllint` to validate YAML files before running playbooks:

```
pip install yamllint
yamllint playbook.yml
```

Ansible also checks syntax with: `ansible-playbook --syntax-check playbook.yml`

Exercise 1: Write Your First YAML File

1. Create a YAML file named `vars.yml` that defines: a string variable `app_name`, an integer `port`, a boolean `ssl_enabled`, and a list of environments (`dev`, `staging`, `prod`).
2. Add a nested mapping called `database` with `host`, `port`, and `name` keys.
3. Add a multi-line literal string under a key called `startup_script` with at least 3 shell commands.
4. Validate the file with: `python3 -c "import yaml; yaml.safe_load(open('vars.yml'))"` and fix any errors.

Exercise 2: YAML Inventory File

1. Create a YAML-format Ansible inventory file (`inventory.yml`) with two groups: `web_servers` (3 hosts) and `db_servers` (1 host).
2. Add `ansible_host`, `ansible_user`, and `ansible_port` variables to each host.
3. Add a group variable `http_port: 80` that applies to all `web_servers`.
4. Add a parent group called `production` containing both `web_servers` and `db_servers`.
5. Verify with: `ansible-inventory -i inventory.yml --list`

Exercise 3: Debug YAML Errors

1. Introduce three deliberate errors in a YAML file (wrong indentation, unquoted special character, missing space after colon).
2. Run `yamllint` against the file and interpret the error messages.
3. Fix each error and re-run `yamllint` to confirm all errors are resolved.

Resources & Further Reading

YAML Specification: <https://yaml.org/spec/1.2-old/spec.html>

Ansible YAML syntax docs:

https://docs.ansible.com/ansible/latest/reference_appendices/YAMLSyntax.html

YAML Lint validator: <https://www.yamllint.com>

`yamllint` tool: <https://yamllint.readthedocs.io>

Learn YAML in Y minutes: <https://learnxinyminutes.com/docs/yaml/>

LAB 2

Using Variables

Simple and complex variables, Ansible facts, magic variables, connection variables, naming rules, and Ansible Vault

Learning Objectives

- Define and use simple and complex variables in playbooks
- Understand Ansible facts and how to access them
- Use magic variables and connection variables effectively
- Apply correct variable naming rules
- Access variables using Jinja2 syntax and understand precedence
- Protect sensitive data using Ansible Vault

1. Simple Variables

Simple variables hold a single scalar value — a string, integer, or boolean. They are defined in a vars block, inventory, or variable files and accessed using Jinja2 double-brace syntax.

```
# Defining simple variables in a playbook
vars:
  app_name: myapp
  http_port: 8080
  debug_mode: false
  max_retries: 3

# Using variables (Jinja2 templating)
tasks:
  - name: Print the application name
    debug:
      msg: "Deploying {{ app_name }} on port {{ http_port }}"

  - name: Set listening port in config
    lineinfile:
      path: /etc/app/config.ini
      line: "port = {{ http_port }}"
```

⚠ Quoting Rule for Variables

When a value STARTS with {{ }}, the entire value must be quoted:

port: "{{ http_port }}" # CORRECT

port: {{ http_port }} # INCORRECT — YAML parse error

Mid-string usage does not require quotes: msg: "App runs on port {{ http_port }}"

2. Complex Variables

Lists

```
vars:
  packages:
    - nginx
    - git
    - python3

tasks:
  - name: Install all packages
    apt:
      name: '{{ packages }}'
      state: present

  # Access by index
  - debug:
      msg: "First package: {{ packages[0] }}"
```

Dictionaries (Mappings)

```
vars:
  database:
    host: db.example.com
    port: 5432
    name: myapp_db
    credentials:
      user: appuser
      password: secret123

tasks:
  # Dot notation
  - debug:
      msg: "DB host is {{ database.host }}"

  # Bracket notation (safer when key names contain special chars)
  - debug:
      msg: "DB name is {{ database['name'] }}"

  # Accessing nested values
  - debug:
      msg: "User: {{ database.credentials.user }}"
```

3. Ansible Facts

Facts are variables that Ansible automatically collects about managed hosts using the setup module. They run at the start of every play (`gather_facts: true` is the default) and are stored under `ansible_facts`.

```
# Disable fact gathering when not needed (speeds up plays)
- hosts: all
  gather_facts: false
```



```

# Manually gather facts when needed
tasks:
  - name: Gather facts
    setup:

# Gather only a subset of facts (much faster)
  - setup:
      gather_subset:
        - network
        - hardware

```

Commonly used facts:

Fact Variable	Description	Example Value
ansible_hostname	Short hostname	web1
ansible_fqdn	Fully qualified domain name	web1.example.com
ansible_os_family	OS family	Debian or RedHat
ansible_distribution	Linux distribution	Ubuntu
ansible_distribution_version	Version number	22.04
ansible_default_ipv4.address	Primary IPv4 address	192.168.1.10
ansible_memory_mb.real.total	Total RAM in MB	8192
ansible_processor_vcpus	Number of vCPUs	4
ansible_mounts	List of mounted filesystems	[[device:..., mount:...]]
ansible_env.HOME	Environment variable	/root

```

# Example: install correct package based on OS family
tasks:
  - name: Install Apache on Debian/Ubuntu
    apt:
      name: apache2
      state: present
      when: ansible_os_family == 'Debian'

  - name: Install Apache on RedHat/CentOS
    yum:
      name: httpd
      state: present
      when: ansible_os_family == 'RedHat'

# Using network facts
- debug:
    msg: "Server IP: {{ ansible_default_ipv4.address }}"

```

Explore Facts Interactively

Run this ad-hoc command to see all facts for a host:

```
ansible hostname -m setup
```

```
ansible hostname -m setup -a 'filter=ansible_distribution*'
```

Then reference any fact path directly in your playbooks.

4. Magic Variables

Magic variables are special built-in variables that Ansible provides automatically. Unlike facts, they are always available regardless of `gather_facts` setting.

Magic Variable	Description
<code>inventory_hostname</code>	Current host's name as defined in inventory
<code>inventory_hostname_short</code>	Hostname up to the first dot
<code>groups</code>	Dictionary of all inventory groups and their hosts
<code>group_names</code>	List of groups the current host belongs to
<code>hostvars</code>	Dictionary of all variables for all inventory hosts
<code>ansible_play_hosts</code>	List of hosts active in the current play
<code>ansible_play_batch</code>	Current batch of hosts being processed
<code>inventory_dir</code>	Directory of the inventory file
<code>playbook_dir</code>	Directory of the running playbook
<code>ansible_version</code>	Dictionary with Ansible version information

```
# Access variables from ANOTHER host using hostvars
tasks:
  - debug:
      msg: "DB server IP is {{ hostvars['db1'].ansible_default_ipv4.address }}"

# Iterate over a group's hosts
  - debug:
      msg: "Web servers: {{ groups['web_servers'] }}"

# Check which groups the current host is in
  - debug:
      msg: "I am in: {{ group_names }}"

# Use inventory_hostname to create host-specific paths
  - name: Create host-specific config
    template:
      src: host_config.j2
      dest: /etc/app/{{ inventory_hostname }}.conf
```

5. Connection Variables

Connection variables control how Ansible connects to and authenticates with managed hosts. They are typically set in inventory files or `host_vars/group_vars` files.

Variable	Purpose	Example
<code>ansible_host</code>	IP or hostname to connect to	192.168.1.10
<code>ansible_port</code>	SSH port number	22
<code>ansible_user</code>	Remote user to connect as	ubuntu
<code>ansible_password</code>	SSH password (use Vault!)	s3cr3t
<code>ansible_ssh_private_key_file</code>	Path to SSH private key	~/.ssh/id_rsa
<code>ansible_connection</code>	Connection plugin	ssh, local, docker
<code>ansible_become</code>	Enable privilege escalation	true
<code>ansible_become_method</code>	Escalation method	sudo, su, pbrun
<code>ansible_become_user</code>	User to become	root
<code>ansible_become_password</code>	sudo password (use Vault!)	s3cr3t
<code>ansible_python_interpreter</code>	Python binary on remote host	/usr/bin/python3

6. Variable Naming Rules

Follow these rules when naming variables to avoid errors and conflicts:

- Only alphanumeric characters and underscores are allowed: `my_var`, `server_port`
- Must start with a letter or underscore — not a number: `_var` or `var1` (valid); `1var` (invalid)
- Case-sensitive: `my_host` and `My_Host` are different variables
- Avoid Python keywords: `type`, `class`, `list`, `dict`, `set`, `id`, `name`
- Avoid Ansible reserved words: `ansible_*`, `hostvars`, `groups`, `inventory_*`
- Use snake_case by convention: `db_host`, `http_port`, `max_retries`

7. Variable Precedence (Lowest to Highest)

When the same variable is defined in multiple places, Ansible uses this precedence order — higher number wins:

Priority	Source
1 (lowest)	Role defaults (roles/myrole/defaults/main.yml)
2	Inventory group_vars/all
3	Inventory group_vars/<group>
4	Inventory host_vars/<host>

5	Playbook group_vars/all
6	Playbook group_vars/<group>
7	Playbook host_vars/<host>
8	Host facts (ansible_facts)
9	Play vars: block
10	Task vars: block
11	set_fact / register
12 (highest)	Extra vars (-e on command line)

8. Protecting Secrets with Ansible Vault

Ansible Vault encrypts sensitive values so they can be safely stored in version control.

```
# Create an encrypted variable file
ansible-vault create group_vars/all/vault.yml

# Edit an existing encrypted file
ansible-vault edit group_vars/all/vault.yml

# Encrypt a single string value (paste into a variable file)
ansible-vault encrypt_string 'mysecretpassword' --name 'db_password'

# View encrypted file contents
ansible-vault view group_vars/all/vault.yml

# Re-key (change vault password)
ansible-vault rekey group_vars/all/vault.yml

# Run playbook and provide vault password interactively
ansible-playbook playbook.yml --ask-vault-pass

# Run playbook with vault password stored in a file
ansible-playbook playbook.yml --vault-password-file ~/.vault_pass
```

Vault Best Practice

Keep vault files separate from plain variable files:

```
group_vars/
  web_servers/
    vars.yml    # plain variables
    vault.yml   # encrypted secrets
```

Prefix vault variables with vault_ to distinguish them:

```
vault_db_password: 's3cr3t'
```

Then in vars.yml reference it:

```
db_password: '{{ vault_db_password }}
```

Exercise 1: Define and Use Variables

1. Create a playbook with a vars block defining: app_name (string), app_port (int), features (list of 3 items), and db_config (dict with host and port).
2. Add tasks that use each variable type — use debug module to print all of them.
3. Access a nested dict value and a list item by index.
4. Run the playbook and verify all values are correctly substituted.

Exercise 2: Explore Ansible Facts

1. Run `ansible localhost -m setup` to view all facts on your local machine.
2. Write a playbook that prints: OS family, distribution, total memory in MB, and primary IP address using facts.
3. Add a task that uses `ansible_os_family` to print a distribution-specific message.

Exercise 3: Use Magic Variables

1. Write a playbook that targets multiple hosts and uses `inventory_hostname` to create per-host log file names.
2. Use hostvars to print the `ansible_host` value of a different host from within one host's task.
3. Print the list of groups your current host belongs to using `group_names`.

Exercise 4: Ansible Vault

1. Create a vault-encrypted file containing `db_password` and `api_key` variables.
2. Reference the encrypted variables in a playbook task using the debug module.
3. Run the playbook using `--ask-vault-pass` and verify the decrypted values are printed.
4. Re-key the vault file with a new password.

Resources & Further Reading

Ansible variables docs:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html

Ansible facts docs:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_vars_facts.html

Magic variables:

https://docs.ansible.com/ansible/latest/reference_appendices/special_variables.html

Ansible Vault: https://docs.ansible.com/ansible/latest/vault_guide/index.html

Variable precedence:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html#understanding-variable-precedence

LAB 3

Ansible Playbooks

Structure, plays, tasks, idempotency, privilege escalation, and handlers

Learning Objectives

- Understand the full structure of an Ansible playbook
- Write plays with hosts, vars, tasks, and handlers
- Implement idempotent tasks using appropriate modules and directives
- Use become for privilege escalation
- Implement handlers that react to task changes
- Execute playbooks with useful command-line options

1. Playbook Structure Overview

An Ansible playbook is a YAML file containing an ordered list of plays. Each play maps a set of tasks to a group of target hosts. Multiple plays in a single file allow you to orchestrate complex multi-tier workflows.

```
---
# Play 1 - configure web servers
- name: Configure web servers
  hosts: web_servers
  become: true
  gather_facts: true
  vars:
    http_port: 80
  vars_files:
    - vars/web_vars.yml
  tasks:
    # tasks go here
  handlers:
    # handlers go here

# Play 2 - configure database servers
- name: Configure database servers
  hosts: db_servers
  become: true
  tasks:
    # tasks go here
---
```

2. Play-Level Directives

Directive	Description	Default
-----------	-------------	---------

hosts	Target hosts or groups (required)	—
name	Human-readable description of the play	unnamed
become	Enable privilege escalation	false
become_user	User to escalate to	root
become_method	Escalation method (sudo, su, pbrun)	sudo
gather_facts	Collect host facts before tasks	true
vars	Inline variable definitions	{}
vars_files	List of external variable files to load	[]
vars_prompt	Prompt user for variable values	[]
tags	Run play only when matching tags given	[]
serial	Number of hosts to process at once	all
max_fail_percentage	Abort play if failure rate exceeds this	0
any_errors_fatal	Stop all hosts on first failure	false
ignore_errors	Continue even if tasks fail	false
environment	Environment variables for all tasks	{}

3. Task Structure

Each task in a play calls one Ansible module with its parameters. Tasks execute in order from top to bottom.

```

tasks:
  - name: Ensure nginx is installed          # Human-readable label (required
    best practice)                          #
    apt:                                    # Module name
      name: nginx                           # Module parameter
      state: present                       # Module parameter
      become: true                          # Override play-level become per
    task                                    #
    when: ansible_os_family == 'Debian'    # Conditional execution
    register: nginx_install_result         # Capture return value
    notify: Restart nginx                  # Trigger handler on change
    ignore_errors: true                   # Continue even if task fails
    tags:                                  # Tag for selective execution
      - packages
      - nginx
    vars:                                  # Task-scoped variables
      my_task_var: value

```

```

loop:                                # Iterate task over list
  - nginx
  - curl
changed_when: false                  # Override change detection
failed_when: result.rc not in [0, 1] # Custom failure condition
no_log: true                         # Hide task output (for secrets)

```

4. Idempotency

Idempotency is the most important principle of Ansible automation. An idempotent operation produces the same result whether run once or a hundred times. Running your playbook multiple times must be safe and produce no unintended changes.

- **Most Ansible modules are idempotent: apt ensures a package is present, not installs it every time**
- command and shell modules are NOT idempotent by default — they always run
- Use `creates:` or `removes:` with `command` to add idempotency
- Use `changed_when: false` when a command never changes state (e.g., read-only queries)
- Use `changed_when` with a condition to accurately detect real changes

```

# NOT idempotent - always runs and always reports 'changed'
- name: Create directory
  command: mkdir /opt/myapp

# IDEMPOTENT - only runs if /opt/myapp does not exist
- name: Create directory
  command: mkdir /opt/myapp
  args:
    creates: /opt/myapp

# BEST - use the file module which is inherently idempotent
- name: Create directory
  file:
    path: /opt/myapp
    state: directory
    mode: '0755'

# changed_when example - always runs but only marks changed on exit code 1
- name: Check for config drift
  command: diff /etc/app/config.ini /tmp/expected.ini
  register: diff_result
  changed_when: diff_result.rc == 1
  failed_when: diff_result.rc > 1

```

5. Privilege Escalation with become

Most system administration tasks require root privileges. The `become` directive instructs Ansible to escalate privileges using `sudo` (default), `su`, or other methods.

```

# Play-level become - applies to all tasks in the play
- hosts: all

```



```

become: true
become_user: root
tasks:
  - name: Install package (runs as root)
    apt:
      name: nginx
      state: present

  - name: Read a user file (run as specific user, overrides play-level)
    command: cat /home/alice/.bashrc
    become_user: alice

  - name: Task without privilege escalation
    command: whoami
    become: false

# Passwordless sudo (recommended): configure in /etc/sudoers
# deploy_user ALL=(ALL) NOPASSWD: ALL

# With sudo password (stored in vault)
- hosts: prod
  become: true
  become_method: sudo
  vars:
    ansible_become_password: '{{ vault_sudo_password }}'

```

6. Handlers

Handlers are special tasks that run only when notified by another task, and only if that task reported a change. Even if a handler is notified multiple times during a play, it runs exactly once at the end — preventing unnecessary restarts.

```

tasks:
  - name: Deploy nginx virtual host config
    template:
      src: vhost.conf.j2
      dest: /etc/nginx/sites-available/myapp.conf
    notify: Reload nginx # Only fires if template changed

  - name: Enable the virtual host
    file:
      src: /etc/nginx/sites-available/myapp.conf
      dest: /etc/nginx/sites-enabled/myapp.conf
      state: link
    notify: Reload nginx # Still only one reload at end

handlers:
  - name: Reload nginx
    service:
      name: nginx
      state: reloaded

  - name: Restart nginx
    service:
      name: nginx
      state: restarted

```

```
# listen: allows multiple tasks to notify using a generic name
- name: Restart application stack
  service:
    name: myapp
    state: restarted
    listen: 'restart stack'          # notify this with 'restart stack'

- name: Restart database
  service:
    name: mysql
    state: restarted
    listen: 'restart stack'
```

Force Handlers to Run Immediately

By default handlers run after all tasks in a play complete.

Use meta: flush_handlers to run pending handlers immediately at that point:

```
- name: Flush handlers now
  meta: flush_handlers
```

7. Running Playbooks

```
# Basic execution
ansible-playbook playbook.yml

# Specify inventory
ansible-playbook -i inventory.ini playbook.yml

# Limit to a subset of hosts
ansible-playbook playbook.yml --limit web1,web2
ansible-playbook playbook.yml --limit 'web_servers:&staging' # intersection

# Run only tasks with specific tags
ansible-playbook playbook.yml --tags 'packages,config'
ansible-playbook playbook.yml --skip-tags 'packages'

# Dry run - show what would change without changing anything
ansible-playbook playbook.yml --check

# Show file diffs in check mode
ansible-playbook playbook.yml --check --diff

# Start from a specific task
ansible-playbook playbook.yml --start-at-task 'Deploy nginx config'

# Verbose output
ansible-playbook playbook.yml -v      # basic
ansible-playbook playbook.yml -vvv    # full debug

# Pass extra variables
ansible-playbook playbook.yml -e 'http_port=8080 env=staging'
ansible-playbook playbook.yml -e '@extra_vars.yml'
```

Exercise 1: Build a Full Playbook

1. Write a playbook that configures a web server: installs nginx, copies a custom index.html, starts and enables the nginx service.
2. Add a handler that reloads nginx when the index.html changes.
3. Use become: true at the play level.
4. Run with --check --diff first, then run for real, then run again to verify idempotency (should show no changes on second run).

Exercise 2: Demonstrate Idempotency

1. Write a task using the command module that creates a file — confirm it is NOT idempotent (runs every time).
2. Fix it by adding creates: so it only runs when the file does not exist.
3. Replace it with the file module and confirm it is fully idempotent.
4. Add changed_when: false to a status-checking command task.

Exercise 3: Handlers with listen

1. Create a playbook with two tasks that both notify a single handler using the listen keyword instead of the handler's name.
2. Trigger both tasks by modifying their source files, and verify the handler runs only once.
3. Add a meta: flush_handlers step in the middle of the task list and observe the difference in execution order.

Resources & Further Reading

Ansible playbooks guide:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_intro.html

Privilege escalation:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_privilege_escalation.html

Handlers: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_handlers.html

Idempotency in Ansible:

https://docs.ansible.com/ansible/latest/reference_appendices/glossary.html

ansible-playbook CLI: <https://docs.ansible.com/ansible/latest/cli/ansible-playbook.html>

LAB 4

Ansible Modules

Module categories, idempotency, return values, and the most commonly used modules

Learning Objectives

- Understand what Ansible modules are and how they execute
- Distinguish between idempotent and non-idempotent modules
- Know the main module categories: system, files, database, cloud/network
- Read and use module return values with the register directive
- Use the most commonly required modules confidently

1. What Are Modules?

Modules are the units of work in Ansible. Each module is a standalone script (usually Python) that Ansible copies to the managed host, executes, and removes. Modules accept parameters as key-value pairs in your task definition and return results as JSON.

- Ansible ships with over 3,000 built-in modules across collections
- Modules are called by name in task definitions (apt:, service:, copy:, etc.)
- Custom modules can be written in Python, Bash, or any language that can output JSON
- Use `ansible-doc <module>` to view documentation and examples locally

```
# View module documentation
ansible-doc apt
ansible-doc -l          # List all available modules
ansible-doc -l | grep file # Search module list

# Run a module ad-hoc (without a playbook)
ansible web1 -m apt -a 'name=nginx state=present' -b
ansible all -m ping
ansible all -m setup -a 'filter=ansible_distribution'
```

2. Module Idempotency

Most Ansible modules implement idempotency internally — they first check the current state, and only take action if the desired state differs. The modules return `changed: true` only when they actually made a change.

Module	Non-Idempotent Behaviour	How to Handle
command	Always runs, always reports changed	Use <code>creates:</code> / <code>removes:</code> args
shell	Always runs, always reports	Use <code>creates:</code> or <code>changed_when:</code>

	changed	
raw	Always runs, always reports changed	Use changed_when: false
script	Always runs, always reports changed	Use creates: or changed_when:
apt/yum	Idempotent by design	Use state: present/absent/latest
service	Idempotent by design	Use state: started/stopped/restarted
copy/template	Idempotent — checksum-based	Nothing extra needed

3. Module Categories

System Modules

Module	Purpose	Key Parameters
user	Manage OS user accounts	name, uid, groups, state, shell, password
group	Manage OS groups	name, gid, state
service	Manage services (SysV, systemd)	name, state, enabled
systemd	Manage systemd units	name, state, enabled, daemon_reload
cron	Manage cron jobs	name, minute, hour, job, state, user
hostname	Set system hostname	name
sysctl	Set kernel parameters	name, value, state, reload
mount	Manage filesystem mounts	path, src, fstype, opts, state
ping	Test connectivity (no params needed)	—
gather_facts / setup	Collect host facts	gather_subset, filter

Files Modules

Module	Purpose	Key Parameters
copy	Copy local file to remote host	src, dest, mode, owner, backup
template	Render Jinja2 template and copy to host	src, dest, mode, owner
file	Manage file/directory attributes	path, state, mode, owner, recurse

	and links	
lineinfile	Manage a single line in a file	path, line, regexp, state, insertafter
blockinfile	Manage a block of lines in a file	path, block, marker, state
fetch	Retrieve a file from remote host	src, dest, flat
find	Find files matching criteria	paths, patterns, age, size, recurse
stat	Get file status / attributes	path
archive	Create tar/gz/bz2 archives	path, dest, format
unarchive	Extract archives	src, dest, remote_src
get_url	Download file from URL	url, dest, mode, checksum
replace	Replace all occurrences of pattern in file	path, regexp, replace, backup

Database Modules

Module	Purpose
mysql_db	Create/drop MySQL databases
mysql_user	Manage MySQL users and privileges
mysql_query	Execute MySQL queries
postgresql_db	Create/drop PostgreSQL databases
postgresql_user	Manage PostgreSQL users and privileges
postgresql_query	Execute PostgreSQL queries
mongodb_user	Manage MongoDB users
redis	Execute Redis commands

Cloud & Network Modules

Category	Modules
AWS	ec2_instance, s3_bucket, rds_instance, iam_user, vpc_net, elb_application_lb
GCP	gcp_compute_instance, gcp_storage_bucket, gcp_container_cluster
Azure	azure_rm_virtualmachine, azure_rm_storageaccount, azure_rm_networkinterface

Network	uri (HTTP), get_url, ios_command, nxos_config, junos_command
Containers	docker_container, docker_image, docker_network, k8s

4. Return Values

Every module returns a set of values that can be captured with register and used in subsequent tasks. Some return values are standard across all modules, while others are module-specific.

Return Key	Type	Always Present?	Description
changed	bool	Yes	True if the module made a change to the system
failed	bool	Yes	True if the module failed
msg	str	Usually	Human-readable result message
rc	int	command/shell	Return code (0 = success)
stdout	str	command/shell	Standard output of the command
stderr	str	command/shell	Standard error of the command
stdout_lines	list	command/shell	stdout split into a list of lines
results	list	With loop:	List of result dicts, one per loop iteration
stat	dict	stat module	File attributes: size, mode, exists, mtime...
ansible_facts	dict	setup module	Facts added to the host's variable space

```
tasks:
  # Capture command output
  - name: Get disk usage
    command: df -h /
    register: disk_result

  - debug:
      msg: |
        Exit code: {{ disk_result.rc }}
        Output: {{ disk_result.stdout }}
        Lines: {{ disk_result.stdout_lines }}

  # Check file existence with stat module
  - name: Check if config exists
    stat:
```

```

    path: /etc/app/config.ini
    register: config_stat

- name: Create config only if missing
  template:
    src: config.ini.j2
    dest: /etc/app/config.ini
    when: not config_stat.stat.exists

# Loop result access
- name: Install multiple packages
  apt:
    name: '{{ item }}'
    state: present
    loop: [nginx, git, curl]
    register: install_results

- debug:
    msg: '{{ install_results.results | map(attribute="item") | list }}'
```

5. Commonly Used Modules — Examples

apt / yum — Package Management

```

# Install a single package
- apt:
    name: nginx
    state: present
    update_cache: true
    cache_valid_time: 3600      # Only update cache if older than 1 hour

# Install multiple packages
- apt:
    name:
      - nginx
      - python3
      - git
    state: present

# Remove a package
- apt:
    name: apache2
    state: absent
    purge: true                # Also remove config files

# Upgrade all packages
- apt:
    upgrade: dist
    update_cache: true
```

template — Jinja2 File Deployment

```

# Task
- name: Deploy nginx virtualhost config
```



```

template:
  src: templates/nginx_vhost.conf.j2
  dest: /etc/nginx/sites-available/{{ app_name }}.conf
  owner: root
  group: root
  mode: '0644'
  backup: true
  notify: Reload nginx

# templates/nginx_vhost.conf.j2
server {
  listen {{ http_port }};
  server_name {{ server_name }};
  root /var/www/{{ app_name }};
  {% if ssl_enabled %}
  listen 443 ssl;
  ssl_certificate /etc/ssl/{{ server_name }}.crt;
  {% endif %}
}

```

user — User Management

```

- name: Create application user
  user:
    name: deploy
    uid: 1500
    groups: sudo,docker
    append: true           # Add to groups, don't replace existing
    shell: /bin/bash
    home: /home/deploy
    create_home: true
    comment: 'Deployment user'
    password: '{{ vault_user_password | password_hash("sha512") }}'
    state: present

```

Exercise 1: File System Modules

1. Create a playbook that: creates a directory /opt/myapp with 0755 permissions, copies a local file into it, creates a symlink from /usr/local/bin/myapp to the copied binary.
2. Add a task that adds a line to /etc/environment only if it does not already exist.
3. Use the stat module to check if a file exists before attempting to delete it.

Exercise 2: Package and Service Modules

1. Write a playbook that installs nginx and git using the appropriate package module for your OS.
2. Start and enable the nginx service using the service module.
3. Use the command module with creates: to test non-idempotent execution, then compare with the file module.

Exercise 3: Template Module

1. Create a Jinja2 template for a simple /etc/motd file that includes the hostname, OS distribution, and current date.
2. Deploy it with the template module and verify the file is rendered correctly on the target host.

Exercise 4: Return Values

1. Use the command module to run `df -h` and capture the output with register.
2. Print only `stdout_lines` from the result using the debug module.
3. Use `failed_when` to make the task fail if the disk is more than 80% full.

Resources & Further Reading

Ansible module index: https://docs.ansible.com/ansible/latest/collections/index_module.html

ansible-doc CLI: <https://docs.ansible.com/ansible/latest/cli/ansible-doc.html>

Return values:

https://docs.ansible.com/ansible/latest/reference_appendices/common_return_values.html

Ansible.Builtin collection: <https://docs.ansible.com/ansible/latest/collections/ansible/builtin/>

Template module: https://docs.ansible.com/ansible/latest/collections/ansible/builtin/template_module.html

LAB 5

Conditionals with Ansible

when statements, Ansible facts, registered variables, boolean operators, and using loops with conditions

Learning Objectives

- Apply the when directive to control task execution
- Use Ansible facts as conditions for OS-specific logic
- Apply registered variable output as conditions
- Combine multiple conditions with boolean and logical operators
- Use conditionals together with loops

1. The when Statement

The when directive makes a task conditional. It accepts a Jinja2 expression (without curly braces) that evaluates to true or false. The task runs only when the expression is true.

```
tasks:
  # Simple string comparison
  - name: Only on Ubuntu
    debug:
      msg: 'This is an Ubuntu server'
    when: ansible_distribution == 'Ubuntu'

  # Numeric comparison
  - name: Warn if low memory
    debug:
      msg: 'WARNING: Less than 2GB RAM'
    when: ansible_memory_mb.real.total < 2048

  # Variable defined check
  - name: Configure optional feature
    template:
      src: feature.conf.j2
      dest: /etc/app/feature.conf
    when: feature_enabled is defined and feature_enabled

  # Membership test
  - name: Install on Debian-family OSes
    apt:
      name: nginx
      state: present
    when: ansible_os_family in ['Debian', 'Ubuntu']
```

2. Conditions Based on Ansible Facts

Facts gathered from the managed host are the most common source of conditional logic — enabling truly cross-platform playbooks.

```
tasks:
  # OS family branching - install correct package
  - name: Install web server (Debian family)
    apt:
      name: apache2
      state: present
    when: ansible_os_family == 'Debian'

  - name: Install web server (RedHat family)
    yum:
      name: httpd
      state: present
    when: ansible_os_family == 'RedHat'

  # Distribution version check
  - name: Apply Ubuntu 22 specific config
    copy:
      src: ubuntu22.conf
      dest: /etc/app/os.conf
    when:
      - ansible_distribution == 'Ubuntu'
      - ansible_distribution_major_version == '22'

  # Hardware-based conditions
  - name: Enable swap only on hosts with less than 4GB RAM
    command: /opt/enable_swap.sh
    when: ansible_memory_mb.real.total < 4096
```

3. Registered Variables in Conditionals

Registering the output of a task and using it as a condition creates dynamic, adaptive playbooks where later tasks react to earlier results.

```
tasks:
  # Check if a service is running
  - name: Check if myapp is running
    command: systemctl is-active myapp
    register: service_status
    ignore_errors: true           # Don't fail if service is not active
    changed_when: false          # Read-only check - never 'changed'

  - name: Start myapp if not running
    service:
      name: myapp
      state: started
    when: service_status.rc != 0

  # Check if file exists before creating it
  - name: Check for existing config
    stat:
      path: /etc/app/config.ini
    register: config_check
```

```

- name: Generate config only if missing
  template:
    src: config.ini.j2
    dest: /etc/app/config.ini
    when: not config_check.stat.exists

# Use stdout content as condition
- name: Check installed version
  command: myapp --version
  register: app_version
  changed_when: false

- name: Upgrade if old version
  apt:
    name: myapp
    state: latest
    when: "'1.0' in app_version.stdout"

```

4. Boolean and Logical Operators

Multiple conditions can be combined using and, or, not, and Python-style comparisons. When when: receives a YAML list, all items are treated as AND conditions.

```

# List format - ALL conditions must be true (AND logic)
when:
  - ansible_os_family == 'Debian'
  - ansible_distribution_major_version | int >= 20
  - http_port is defined

# Inline AND
when: ansible_os_family == 'Debian' and http_port == 80

# OR logic
when: ansible_distribution == 'Ubuntu' or ansible_distribution == 'Debian'

# NOT
when: not ansible_check_mode

# is defined / is not defined
when: my_variable is defined
when: my_variable is not defined

# is none
when: db_password is not none

# Type casting with Jinja2 filters
when: some_string | int > 100
when: flag_value | bool

# in / not in
when: inventory_hostname in groups['web_servers']
when: ansible_distribution not in ['CentOS', 'RHEL']

# is match (regex)
when: inventory_hostname is match('web[0-9]+')

```

5. Conditionals with Loops

A when condition applies to each iteration of a loop independently. The item variable is available inside the when expression, allowing fine-grained control over which loop items are processed.

```
vars:
  services:
    - { name: nginx, enabled: true }
    - { name: apache2, enabled: false }
    - { name: mysql, enabled: true }

tasks:
  # when is evaluated for each item - only enabled services are started
  - name: Start enabled services
    service:
      name: '{{ item.name }}'
      state: started
      loop: '{{ services }}'
      when: item.enabled

  # Skip specific items in a loop
  - name: Install packages except on minimal installs
    apt:
      name: '{{ item }}'
      state: present
      loop: [vim, htop, curl, wget]
      when:
        - item != 'htop' or not minimal_install | default(false)
        - ansible_os_family == 'Debian'
```

Exercise 1: OS-Specific Tasks

1. Write a playbook that installs the correct web server (apache2 on Debian/Ubuntu, httpd on RedHat/CentOS) and configures the appropriate service name, all using when based on `ansible_os_family`.
2. Add a task that only runs on hosts with more than 4GB of RAM.

Exercise 2: Registered Variable Conditions

1. Check if a directory `/opt/myapp` exists using the `stat` module.
2. Use the registered result to create the directory only if it does not exist, and print a skip message if it already exists.
3. Run a version check command and use its `stdout` to decide whether to run an upgrade task.

Exercise 3: Logical Operators

1. Write a task that runs only when: the OS is Ubuntu AND the version is 22 or higher AND a variable `deploy_env` equals 'production'.

2. Add a separate task using OR logic that runs on either Debian or Ubuntu.
3. Create a task that runs only when a variable is NOT defined.

Exercise 4: Loop with when

1. Define a list of dict items where each has a name and an active boolean field.
2. Use loop to iterate and when: item.active to only process the active items.
3. Add a debug task that prints a skip message for items where active is false.

Resources & Further Reading

Ansible conditionals docs:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_conditionals.html

Jinja2 tests reference: <https://jinja.palletsprojects.com/en/3.1.x/templates/#tests>

Ansible facts:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_vars_facts.html

Registered variables:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html#registering-variables

LAB 6

Loops

loop keyword, loop_control options: label, pause, index_var, loop_var, and advanced loop patterns

Learning Objectives

- Use the loop keyword to iterate tasks over lists and dictionaries
- Apply loop_control to customize loop behaviour and output
- Use label to produce readable loop output
- Add pause between loop iterations
- Track iteration position with index_var
- Rename the loop variable with loop_var to avoid conflicts in nested loops

1. The loop Keyword

The loop keyword replaces the older with_items, with_list, and other with_* directives. It iterates a task over a list, making each element available as the item variable.

```
# Simple list loop
- name: Install required packages
  apt:
    name: '{{ item }}'
    state: present
  loop:
    - nginx
    - git
    - python3-pip
    - curl

# Loop using a variable
vars:
  web_packages: [nginx, certbot, python3-certbot-nginx]

tasks:
  - apt:
      name: '{{ item }}'
      state: present
      loop: '{{ web_packages }}'

# Loop over list of dicts
- name: Create system users
  user:
    name: '{{ item.name }}'
    uid: '{{ item.uid }}'
    groups: '{{ item.groups }}'
    state: present
  loop:
    - { name: alice, uid: 1001, groups: sudo }
```



```
- { name: bob,    uid: 1002, groups: docker }
- { name: carol, uid: 1003, groups: www-data }
```

2. Looping Over Dictionaries

To loop over the keys and values of a dictionary, use the `dict2items` Jinja2 filter, which converts the dict into a list of `{key, value}` items.

```
vars:
  nginx_configs:
    worker_processes: auto
    worker_connections: 1024
    keepalive_timeout: 65

tasks:
  - name: Set nginx config values
    lineinfile:
      path: /etc/nginx/nginx.conf
      regexp: '^{{ item.key }}'
      line: '{{ item.key }} {{ item.value }};'
      loop: '{{ nginx_configs | dict2items }}'

  # Access item.key and item.value
  - debug:
      msg: 'Key: {{ item.key }} Value: {{ item.value }}'
      loop: '{{ nginx_configs | dict2items }}'
```

3. loop_control

`loop_control` is a sub-directive of a looping task that gives you fine-grained control over how loops run and how their output is displayed.

label — Customize Output

By default, Ansible prints the entire item value for each iteration — which can be very verbose for complex dicts. The `label` option specifies what to display instead.

```
- name: Create users
  user:
    name: '{{ item.name }}'
    uid: '{{ item.uid }}'
    shell: '{{ item.shell }}'
    home: '{{ item.home }}'
    groups: '{{ item.groups | join(",") }}'
  loop: '{{ users }}'
  loop_control:
    label: '{{ item.name }}'      # Only show name, not entire dict

# Without label - output is messy:
# TASK [Create users] ***
# ok: [host] => (item={'name': 'alice', 'uid': 1001, 'shell': '/bin/bash', ...})
```

```
# With label: '{{ item.name }}' - clean:
# TASK [Create users] ***
# ok: [host] => (item=alice)
```

pause — Delay Between Iterations

The `pause` option adds a wait (in seconds) between each loop iteration. Useful when deploying to services that need time to stabilise before the next item is processed.

```
- name: Restart services with cooldown
  service:
    name: '{{ item }}'
    state: restarted
  loop:
    - app-worker-1
    - app-worker-2
    - app-worker-3
  loop_control:
    pause: 5          # Wait 5 seconds between each restart
    label: '{{ item }}'

# Also useful when hitting rate-limited APIs
- name: Create cloud resources
  uri:
    url: 'https://api.example.com/resources'
    method: POST
    body: '{ "name": "{{ item }}" }'
  loop: '{{ resource_names }}'
  loop_control:
    pause: 2          # Avoid hitting API rate limits
```

index_var — Track Iteration Index

The `index_var` option creates a variable that holds the current iteration number (starting from 0). This is useful when you need to number items or make index-based decisions.

```
- name: Configure app replicas with unique ports
  template:
    src: replica.conf.j2
    dest: '/etc/app/replica-{{ replica_index }}.conf'
  vars:
    replica_port: '{{ 8080 + replica_index }}'
  loop: '{{ replicas }}'
  loop_control:
    index_var: replica_index    # 0, 1, 2, ...
    label: '{{ item.name }}'

# replica.conf.j2 can use replica_index and replica_port:
# [replica]
# id = {{ replica_index + 1 }}
# port = {{ replica_port }}
# name = {{ item.name }}

# Practical: add a numbered suffix to created files
- name: Create numbered backup files
  copy:
```

```

    content: '{{ item }}\n'
    dest: '/tmp/backup-{{ idx + 1 }}.txt'
  loop: ['config1', 'config2', 'config3']
  loop_control:
    index_var: idx

```

loop_var — Rename the Loop Variable

By default, each loop iteration's value is accessible as `item`. When nesting loops (using `include_tasks`), both the outer and inner loop use `item`, causing a collision. `loop_var` renames one of them to avoid the conflict.

```

# outer_loop.yml - outer loop uses 'server' instead of 'item'
- name: Process servers
  include_tasks: inner_loop.yml
  loop: '{{ servers }}'
  loop_control:
    loop_var: server      # item is now called 'server'
    label: '{{ server.name }}'

# inner_loop.yml - inner loop safely uses 'item'
- name: Deploy configs to {{ server.name }}
  template:
    src: '{{ item.template }}'
    dest: '/etc/{{ server.name }}/{{ item.dest }}'
  loop: '{{ server.configs }}' # item is used here without conflict
  loop_control:
    label: '{{ item.dest }}'

# Variables used:
# servers:
#   - name: web1
#     configs:
#       - { template: nginx.j2, dest: nginx.conf }
#       - { template: ssl.j2, dest: ssl.conf }

```

4. Extended Loop Information

Setting `extended: true` under `loop_control` gives access to the `ansible_loop` special variable with useful metadata about the current iteration.

```

- name: Process items with full loop metadata
  debug:
    msg: |
      Item: {{ item }}
      Index (0-based): {{ ansible_loop.index0 }}
      Index (1-based): {{ ansible_loop.index }}
      Reverse index:   {{ ansible_loop.revindex }}
      Total items:     {{ ansible_loop.length }}
      Is first:        {{ ansible_loop.first }}
      Is last:         {{ ansible_loop.last }}
  loop: [alpha, beta, gamma, delta]
  loop_control:
    extended: true

```

```
# Use first/last for special handling
- name: Configure load balancer
  template:
    src: lb_backend.j2
    dest: /etc/lb/backends.conf
  loop: '{{ backend_servers }}'
  loop_control:
    extended: true
    label: '{{ item }}'
  vars:
    is_last_item: '{{ ansible_loop.last }}'
```

5. Loop with register

When a looping task uses register, the result variable contains a results key — a list of result dicts, one for each loop iteration.

```
- name: Check service statuses
  command: systemctl is-active {{ item }}
  loop: [nginx, mysql, redis]
  register: service_checks
  ignore_errors: true
  changed_when: false

# service_checks.results is a list of 3 result dicts
# Each has: item, rc, stdout, stderr, changed, failed

- name: Show services that are NOT running
  debug:
    msg: '{{ item.item }} is NOT running (rc={{ item.rc }})'
  loop: '{{ service_checks.results }}'
  when: item.rc != 0
  loop_control:
    label: '{{ item.item }}'
```

Exercise 1: Loop Over Lists and Dicts

1. Create a playbook that installs 5 packages using loop.
2. Create 3 OS users using loop over a list of dicts with name, uid, and shell fields.
3. Set multiple sysctl values by looping over a dictionary using dict2items.

Exercise 2: loop_control: label and pause

1. Rewrite the user creation loop from Exercise 1 to use loop_control: label so only the username is shown in output.
2. Create a loop that restarts 3 services with a 3-second pause between each restart.
3. Verify the pause is working by checking the timestamps in ansible output.

Exercise 3: index_var

1. Create a loop that generates 5 configuration files named app-1.conf through app-5.conf, using index_var to generate the numbers.
2. Use the index to assign unique port numbers to each file (starting from 8080).

Exercise 4: Nested Loops with loop_var

1. Create two lists: servers (web1, web2) and configs (app.conf, ssl.conf).
2. Use include_tasks with an outer loop (loop_var: server) to iterate over servers.
3. Inside the included task, iterate over configs (using default item variable) to deploy each config to each server.

Resources & Further Reading

Ansible loops docs:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_loops.html

loop_control:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_loops.html#adding-controls-to-loops

Jinja2 filters (dict2items, etc.):

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_filters.html

Migrating from with_* to loop:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_loops.html#migrating-from-with-x-to-loop

LAB 7

Ansible Roles

Structure, creation, usage, defaults vs vars, dependencies, and Ansible Galaxy

Learning Objectives

- Understand the directory structure and purpose of each role component
- Create roles using `ansible-galaxy init`
- Write tasks, handlers, templates, and variables within a role
- Distinguish between role defaults and role vars
- Define role dependencies in `meta/main.yml`
- Use roles in playbooks and share them via Ansible Galaxy

1. What Are Roles?

A role is a standardised, reusable unit of Ansible automation. Instead of placing all tasks, handlers, variables, and templates in a single playbook, roles split them into a structured directory tree. This promotes reusability, testability, and collaboration — you can share a role across teams, projects, or publish it to Ansible Galaxy for the community.

- Roles encapsulate all logic for a single concern (e.g., a 'nginx' role handles everything needed to configure nginx)
- Roles are auto-discovered by Ansible when they follow the standard directory convention
- Roles make large playbooks manageable by replacing hundreds of tasks with a single roles: reference
- Ansible Galaxy hosts thousands of community roles you can install and use immediately

2. Role Directory Structure

The following is the complete directory structure of an Ansible role. Each subdirectory has a specific purpose. Only the `tasks/` directory is required; all others are optional.

```
roles/
  nginx/                                # Role name
    tasks/                              # REQUIRED: entry point, all tasks defined here
      main.yml
    handlers/
      main.yml                          # Handlers (e.g., restart nginx)
    templates/
      nginx.conf.j2                    # Jinja2 templates (accessible via template
module)
      vhost.conf.j2
    files/
      ssl_dhparam.pem                 # Static files (accessible via copy module)
    vars/
```

```

    main.yml                                # HIGH-precedence variables (role internals)
    defaults/
      main.yml                              # LOW-precedence variables (meant to be
overridden)
    meta/
      main.yml                              # Role metadata: author, license, dependencies
    library/
      custom_module.py                      # Custom modules bundled with the role
    module_utils/
      helper.py                             # Utilities for custom modules
    lookup_plugins/
      custom_lookup.py                     # Custom lookup plugins
    README.md                              # Role documentation

```

File Resolution in Roles

Inside a role's tasks, files and templates are referenced by relative path only — no need to include `roles/nginx/files/`:

```

copy: src: ssl_dhparam.pem    # Ansible finds roles/nginx/files/ssl_dhparam.pem
template: src: nginx.conf.j2 # Ansible finds roles/nginx/templates/nginx.conf.j2

```

3. Creating a Role with ansible-galaxy init

```

# Create a new role skeleton
ansible-galaxy init nginx
ansible-galaxy init --init-path roles/ nginx    # Create inside a roles/ directory

# Result: fully scaffolded directory tree is created automatically
# tree roles/nginx
# nginx/
# |— README.md
# |— defaults/main.yml
# |— files/
# |— handlers/main.yml
# |— meta/main.yml
# |— tasks/main.yml
# |— templates/
# |— tests/
# |   |— inventory
# |   |— test.yml
# |— vars/main.yml

```

4. Writing Role Components

tasks/main.yml

```

---
# roles/nginx/tasks/main.yml

- name: Install nginx
  apt:

```

```

    name: nginx
    state: present
    update_cache: true
    notify: Restart nginx

- name: Ensure nginx config directory exists
  file:
    path: /etc/nginx/sites-available
    state: directory
    mode: '0755'

- name: Deploy nginx main configuration
  template:
    src: nginx.conf.j2          # Resolved from roles/nginx/templates/
    dest: /etc/nginx/nginx.conf
    owner: root
    mode: '0644'
  notify: Reload nginx

- name: Copy SSL DH parameters
  copy:
    src: ssl_dhparam.pem        # Resolved from roles/nginx/files/
    dest: /etc/nginx/ssl_dhparam.pem
  when: nginx_ssl_enabled

- name: Start and enable nginx
  service:
    name: nginx
    state: started
    enabled: true

```

handlers/main.yml

```

---
# roles/nginx/handlers/main.yml

- name: Restart nginx
  service:
    name: nginx
    state: restarted

- name: Reload nginx
  service:
    name: nginx
    state: reloaded

```

defaults/main.yml vs vars/main.yml

Roles have two places to define variables with critically different precedence levels:

File	Precedence	Purpose	Can Be Overridden?
defaults/main.yml	Lowest of all sources	Sensible defaults meant for users to change	Yes — by any variable source
vars/main.yml	High (above inventory)	Role-internal constants	Only by play/task vars

		not meant to be changed	or -e
--	--	-------------------------	-------

```
# roles/nginx/defaults/main.yml - USER-CONFIGURABLE defaults
---
nginx_port: 80
nginx_ssl_enabled: false
nginx_worker_processes: auto
nginx_keepalive_timeout: 65
nginx_user: www-data

# roles/nginx/vars/main.yml - ROLE-INTERNAL constants
---
nginx_conf_path: /etc/nginx/nginx.conf
nginx_sites_available: /etc/nginx/sites-available
nginx_sites_enabled: /etc/nginx/sites-enabled
nginx_supported_os_families: ['Debian', 'Ubuntu']
```

5. Using Roles in Playbooks

```
# Method 1: roles: key (classic - roles run before tasks:)
- name: Configure web servers
  hosts: web_servers
  become: true
  roles:
    - nginx                # Shorthand: just the role name
    - role: nginx          # Full syntax (same result)
    - role: nginx          # Override defaults by passing vars
  vars:
    nginx_port: 8080
    nginx_ssl_enabled: true

# Method 2: import_role (static - evaluated at parse time, supports tags)
tasks:
  - name: Configure nginx
    import_role:
      name: nginx
    vars:
      nginx_port: 8080

# Method 3: include_role (dynamic - evaluated at runtime, supports conditionals)
tasks:
  - name: Configure nginx if needed
    include_role:
      name: nginx
    when: install_nginx | default(true)
```

6. Role Dependencies

A role can declare that it depends on other roles in its meta/main.yml. Ansible automatically runs all dependency roles before the role that declares them.

```
# roles/nginx/meta/main.yml
---
galaxy_info:
  author: yourname
  description: Installs and configures nginx
  license: MIT
  min_ansible_version: '2.9'
  platforms:
    - name: Ubuntu
      versions: [focal, jammy]
  galaxy_tags: [web, nginx, proxy]

dependencies:
  - role: common          # Run 'common' role before this role
  - role: ssl_certs        # Run 'ssl_certs' role before this role
  vars:
    cert_domain: '{{ nginx_server_name }}' # Pass vars to dependency
  - role: geerlingguy.firewall # Galaxy role dependency
  vars:
    firewall_allowed_tcp_ports: [80, 443]
```

7. Ansible Galaxy

Ansible Galaxy is the public hub for sharing Ansible roles and collections. The `ansible-galaxy` CLI installs roles locally for use in your playbooks.

```
# Install a role from Galaxy
ansible-galaxy install geerlingguy.nginx
ansible-galaxy install geerlingguy.nginx --force # Re-install/update

# Install to a specific path
ansible-galaxy install geerlingguy.nginx -p roles/

# Install from a requirements.yml file (recommended for teams)
ansible-galaxy install -r requirements.yml

# List installed roles
ansible-galaxy list

# requirements.yml - define all role dependencies
---
roles:
  - name: geerlingguy.nginx
    version: '3.2.0'
  - name: geerlingguy.mysql
    version: '4.0.0'
  - src: https://github.com/myorg/myrole.git
    name: myrole
    version: main
```

8. Role Project Layout

A well-organized Ansible project that uses roles looks like this:

```

ansible-project/
├── site.yml                # Master playbook – calls all roles
├── webservers.yml          # Playbook for web servers
├── dbservers.yml           # Playbook for database servers
├── inventory/
│   ├── production
│   └── staging
├── group_vars/
│   ├── all.yml
│   └── web_servers.yml
├── host_vars/
│   └── web1.yml
├── roles/
│   ├── common/             # Base role applied to all hosts
│   ├── nginx/              # Web server role
│   ├── mysql/              # Database role
│   └── myapp/               # Application deployment role
└── requirements.yml        # External Galaxy roles

```

Exercise 1: Create Your First Role

1. Run `ansible-galaxy init roles/webserver` to create the role skeleton.
2. Add tasks to install nginx, start and enable the service, and deploy a template-based `index.html`.
3. Add a handler to reload nginx when the `index.html` template changes.
4. Add defaults for `http_port` (80) and `server_name` (localhost).
5. Use the role in a playbook and override `http_port`: 8080 via role vars.

Exercise 2: defaults vs vars

1. In your webserver role, define `http_port` in `defaults/main.yml`.
2. Override it from the playbook's role vars: `block`. Confirm the override takes effect.
3. Move it to `vars/main.yml` and try to override it from inventory. Confirm the override no longer works.

Exercise 3: Role Dependencies

1. Create a second role called `common` that installs basic utilities (`curl`, `vim`, `git`) and sets the `timezone`.
2. Add the `common` role as a dependency in your webserver role's `meta/main.yml`.
3. Run the webserver role and confirm that `common` tasks execute first.

Exercise 4: Ansible Galaxy

1. Create a `requirements.yml` that references `geerlingguy.nginx` from Ansible Galaxy.
2. Install it with `ansible-galaxy install -r requirements.yml`.
3. Use the installed role in a playbook and override one of its default variables.

Resources & Further Reading

Ansible roles docs:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html

ansible-galaxy CLI: <https://docs.ansible.com/ansible/latest/cli/ansible-galaxy.html>

Ansible Galaxy hub: <https://galaxy.ansible.com>

Role dependencies:

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html#role-dependencies

Best practices: https://docs.ansible.com/ansible/latest/tips_tricks/ansible_tips_tricks.html

Molecule (role testing): <https://molecule.readthedocs.io>